2026年度

機械学習演習

（第1回：ガイダンス・環境構築・基礎知識）

計良 宥志

本日の内容

1. **授業概要** — シラバス・成績評価・進め方
2. **計算環境の設定** — エディタ・Git・conda
3. **必要な事前知識** — 線形代数と最適化の復習
4. **機械学習の勉強のコツ** — 概念の分解と理解の階層

1. 授業概要

授業概要

目標：機械学習Iの各項目に関連し，発展と実装で理解を深める

成績

- Moodle上での小テスト・実装の提出（予定）

事前知識

- 基本的な線形代数，最適化に関する復習
- Python等の基本的なプログラミング

注意

- 自身のPC・電源アダプターを持参
- 前回の機械学習Iの内容を復習
- **生成AI**：趣旨を損なう使い方禁止（コード**生成**は原則 NG・補完・作図用は可）

2. 計算環境の設定

概要：セットアップする3つのもの

#	ツール	用途
1	**Cursor** または **VS Code + GitHub Copilot**	コードエディタ + AI補助
2	**Git / GitHub**	バージョン管理・コード共有
3	**conda** (Anaconda)	Python仮想環境の管理

上2つは学生向け無料枠あり → 各自申請

Anaconda のインストール

1. anaconda.com からインストーラをダウンロード
2. 自分のOSに合ったものを選択（Windows / macOS / Linux）
3. インストーラの指示に従いインストール
4. ターミナル（macOS/Linux）またはAnaconda Prompt（Windows）を開く

```
# インストール確認
conda --version
```

Cursor / VS Code + GitHub Copilot

できること

- **補完**：続きの予測入力
- **チャット**：エラー・書き方の相談
- **説明・書き換え**：選択範囲
- **ターミナル / Git**：エディタ内で操作
- **拡張**：Jupyter, Python, LaTeX ほか

Git / GitHub：なぜバージョン管理が必要か？

次のような経験はないだろうか

- `report_final.py`，`report_final2.py`，`report_本当のfinal.py` ……といったファイル名の乱立
- 「動いてたコードをいじったら壊れた．元に戻せない」
- 「チームで同じファイルを編集したら上書きされた」
- 「先週のバージョンに戻したいけど，どれだっけ？」
- 「このコード誰が何のために変えたんだっけ？」

> **Git**：変更の記録・巻き戻し・共有

Git の基本的な使い方

```
# リポジトリの作成
git init

# 変更をステージング → コミット（記録）
git add .
git commit -m "変更内容のメッセージ"

# 変更履歴を確認
git log --oneline

# リモート（GitHub）にプッシュ
git push origin main
```

GitHub に自分用リポを作り，演習コードを管理

Git のインストール（一般的な入れ方）

Git は PC に入れる（conda の Python 環境とは別）

まず：有無の確認

ターミナル / PowerShell / cmd で：

```
git --version
```

- バージョン表示（例 `git version 2.43.0`）→ **利用可** → 以降のインストール手順は不要
- `command not found` 等 → **未導入** → 次ページの OS 別手順へ

macOS（未導入のとき）

- 多くは利用可だが，未セットアップもある
- `git --version` の案内に従う，または `xcode-select --install`
- `/usr/bin/git` がスタブで CLT 導入を促す場合あり

Windows（未導入のとき）

- Git for Windows
- 再実行： `git --version`

Linux（例：Ubuntu）（未導入のとき）

- `sudo apt update && sudo apt install git` → `git --version`

再確認

`git clone` 等の前に **`git --version`**

conda・仮想環境：なぜ必要か？

次のような経験はないだろうか

- 「`pip install` したらPCの別のプログラムが動かなくなった」
- 「Aの課題では numpy 1.x，Bの課題では numpy 2.x が必要…」
- 「友達のコードを自分のPCで実行するとエラーになる」
- 「先月動いたスクリプトが今日はなぜか動かない」
- 「OSのPythonを壊してしまって色々大変なことに…」

> **プロジェクトごと独立した Python 環境**（他と干渉しない）

conda による仮想環境の作り方・使い方

```
# リポジトリのルート（MachineLearningExercise/）で実行
conda env create -f environment.yml

# 環境の有効化
conda activate ml-exercise

# 環境の一覧確認
conda env list

# 環境の無効化
conda deactivate
```

作業

1. **Git**： `git --version` → なければ導入（前スライド）
2. **Anaconda** のインストール（ `conda` を含む）
3. **Cursor** のインストール（または VS Code と GitHub Copilot 拡張）
4. **GitHub** アカウントの開設
5. **自分用の空リポジトリ**を GitHub 上で作成（名前は任意・**README なし**が無難）
6. **授業リポジトリのクローン**：
7. `conda` 環境の作成
8. **自分のリポジトリへ push**

作業：ターミナルでのコマンド例

```
# 作業ディレクトリの例（任意のパスでよい）
mkdir -p ~/study/2026/機械学習演習
cd ~/study/2026/機械学習演習

# --- 6. clone ---
git clone https://github.com/HiroshiKERA/MachineLearningExercise.git
cd MachineLearningExercise

# --- 7. conda ---
conda env create -f environment.yml
conda activate ml-exercise

# --- 8. push（講義 = upstream, 自分 = origin）---
git remote rename origin upstream
git remote add origin https://github.com/<yourname>/<your-repo>.git
git remote -v
git push -u origin main
```

講義資料が更新されたとき

```
git fetch upstream
git merge upstream/main
# 競合を直したら
git push origin main
```

日々の作業の流れ（例）

```
# conda環境の起動
conda activate ml-exercise

# 適当の作業の区切りごと
git add .
git commit -m "<更新ログ>"

# GitHubへのpush（必要に応じて）
git push origin main
```

3. 必要な事前知識

必要な事前知識

前提：**Python，線形代数・最適化**

機械学習の三要素

比喩	実体	説明
教材に相当するもの	**データセット**	モデルが学ぶための材料
学習する主体に相当するもの	**AIモデル**	データからパターンを学ぶ主体
指導に相当するもの	**最適化**	モデルを改善する仕組み

- **AIモデルの理解** ← 線形代数（計算のフローを追う）
- **最適化の理解** ← 微分・最適化理論

線形代数が必要な例 (1)：MLPの計算フロー

Multi-Layer Perceptron（多層パーセプトロン）

入力 $x \in \mathbb{R}^d$ に対して：

$$h_1 = \sigma(W_1 x + b_1), \quad h_2 = \sigma(W_2 h_1 + b_2), \quad \hat{y} = W_3 h_2 + b_3$$

- 各層は **行列の積** → **バイアス加算** → **活性化関数** の繰り返し
- $W_1 \in \mathbb{R}^{m_1 \times d}$, $W_2 \in \mathbb{R}^{m_2 \times m_1}$, $W_3 \in \mathbb{R}^{o \times m_2}$
- $b_1 \in \mathbb{R}^{m_1}$, $b_2 \in \mathbb{R}^{m_2}$, $b_3 \in \mathbb{R}^{o}$, $\sigma : \mathbb{R} \to \mathbb{R}$

> 線形代数に慣れると **行列の連鎖** と見える

線形代数が必要な例 (2)：Attention の計算フロー

Attention（注意機構） — Transformer の中核

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $Q \in \mathbb{R}^{N_q \times d_k}$, $K \in \mathbb{R}^{N_k \times d_k}$, $V \in \mathbb{R}^{N_k \times d_v}$
- $QK^\top$：全てのQuery–Keyペアの類似度を**行列積**で一括計算
- softmax で正規化 → V と加重和

> 大規模でも流れは **線形代数の組み合わせ**

最適化の基礎 (1)：スカラー関数のベクトル微分

スカラー値関数 $f : \mathbb{R}^n \to \mathbb{R}$ の勾配を
$\nabla f(x) \in \mathbb{R}^n$ と書く.

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix} \in \mathbb{R}^n$$

演習：ベクトル微分

以下の関数 f の勾配 $\nabla f(x)$ を求めよ.

1. $x, c \in \mathbb{R}^n$: $f(x) = c^\top x$
2. $x \in \mathbb{R}^n$: $f(x) = \|x\|_2^2 = x^\top x$
3. $x \in \mathbb{R}^n,\ M \in \mathbb{R}^{n \times n}$ (対称) : $f(x) = x^\top M x$

最適化の基礎 (2)：Lagrange の未定乗数法

等式制約付きの最適化問題

$$\max_{x \in \mathbb{R}^n} f(x) \quad \text{s.t.} \quad g(x) = 0, \quad g : \mathbb{R}^n \to \mathbb{R}$$

> **制約つきの最適化問題を制約なしの問題にしたい**

Lagrangian を導入：

$$\mathcal{L}(x, \lambda) = f(x) - \lambda g(x), \quad (x, \lambda) \in \mathbb{R}^n \times \mathbb{R}$$

必要条件（停留条件）：

$$\nabla_x \mathcal{L}(x, \lambda) = 0, \quad \partial_\lambda \mathcal{L}(x, \lambda) = 0$$

例題

$A \in \mathbb{R}^{n \times n}$ を対称行列とし，次を解け.

$$\max_{x \in \mathbb{R}^n} x^\top A x \quad \text{s.t.} \quad \|x\|_2 = 1$$

--- -->

4. 機械学習の勉強のコツ

概念を分解して理解する

用語・概念は多い → **階層に分解**して整理

分解

1. **どのレイヤーの概念か**（上位・下位・関連概念は？）
2. **何のためのものか**（何をするのに便利？）
3. **そのアイディア**
4. **具体イメージ（入力・出力・制約・定義など）**
5. 計算の流れ・原理

ポイント

- **ある階層で理解を打ち切る（全部把握しようとしない）**
- **具体例に触れる（計算・実装）**

例：微分

例：リプシッツ連続

1. 関数の連続性・滑らかさに関連する
2. 微分可能でない関数に関して，「変化の大きさ」を考えたい
3. 差分で **傾き** の代用
4. $f : \mathbb{R} \to \mathbb{R}$ に定数 $L > 0$ が存在し，任意の $x, y \in \mathbb{R}$ で次が成り立つならば，**fは L-リプシッツ連続**.

$$|f(x) - f(y)| \leq L|x - y|$$

演習1：固有値分解

演習2：Lagrange の未定乗数法

まとめ

課題

- 実験環境を設定すること
- 自分のGitHubのリポジトリにpush
- 「概念を分解して理解」の演習2つをMoodle上で提出.

次回

- 機械学習I 第1回に対応した演習